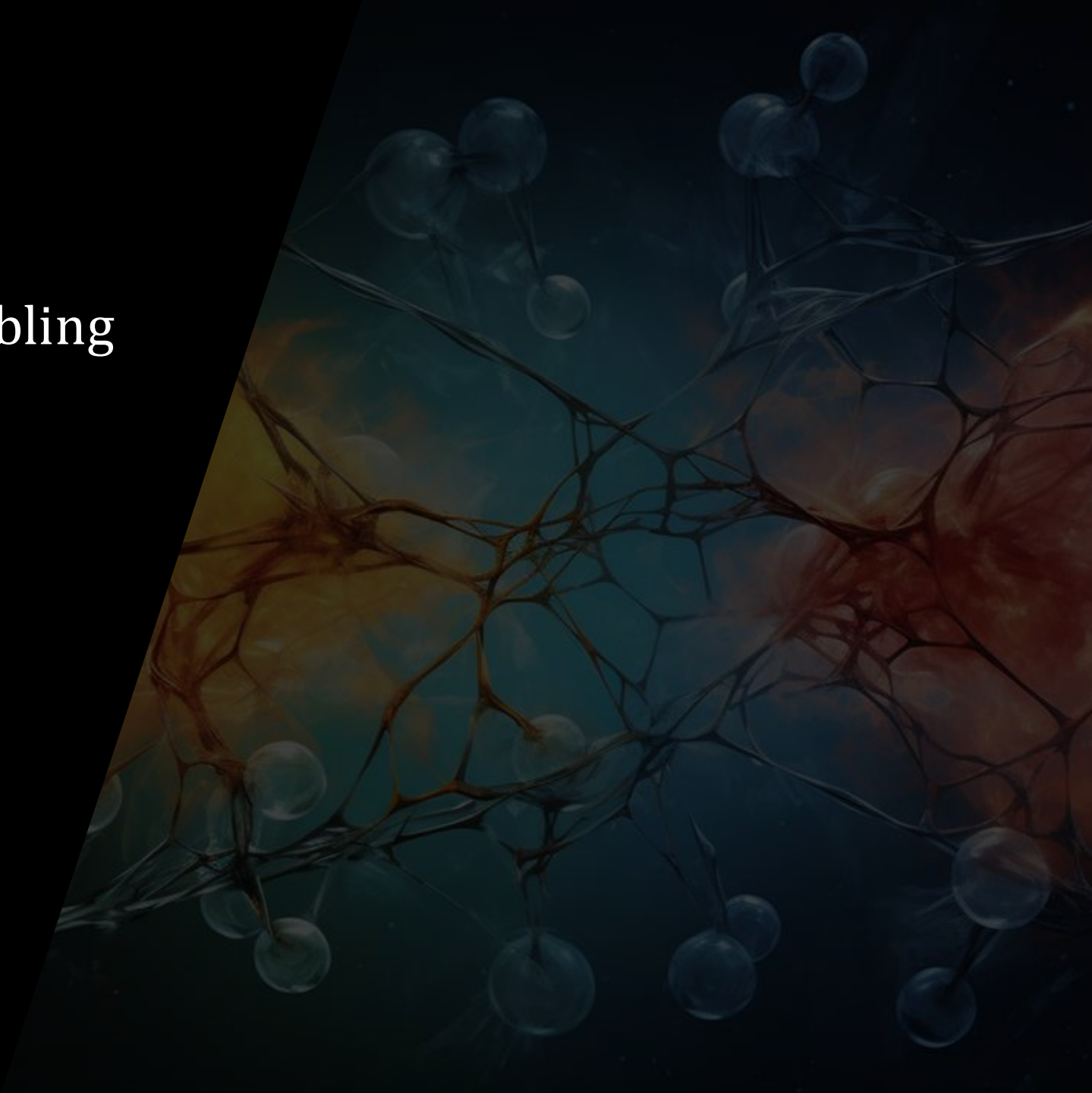


Session 5.2 – Trees and Ensembling methods

Contents:

- *A first non-linear model: decision trees*
- *Ensembling: bagging and boosting*
- *Random forest and boosted trees*



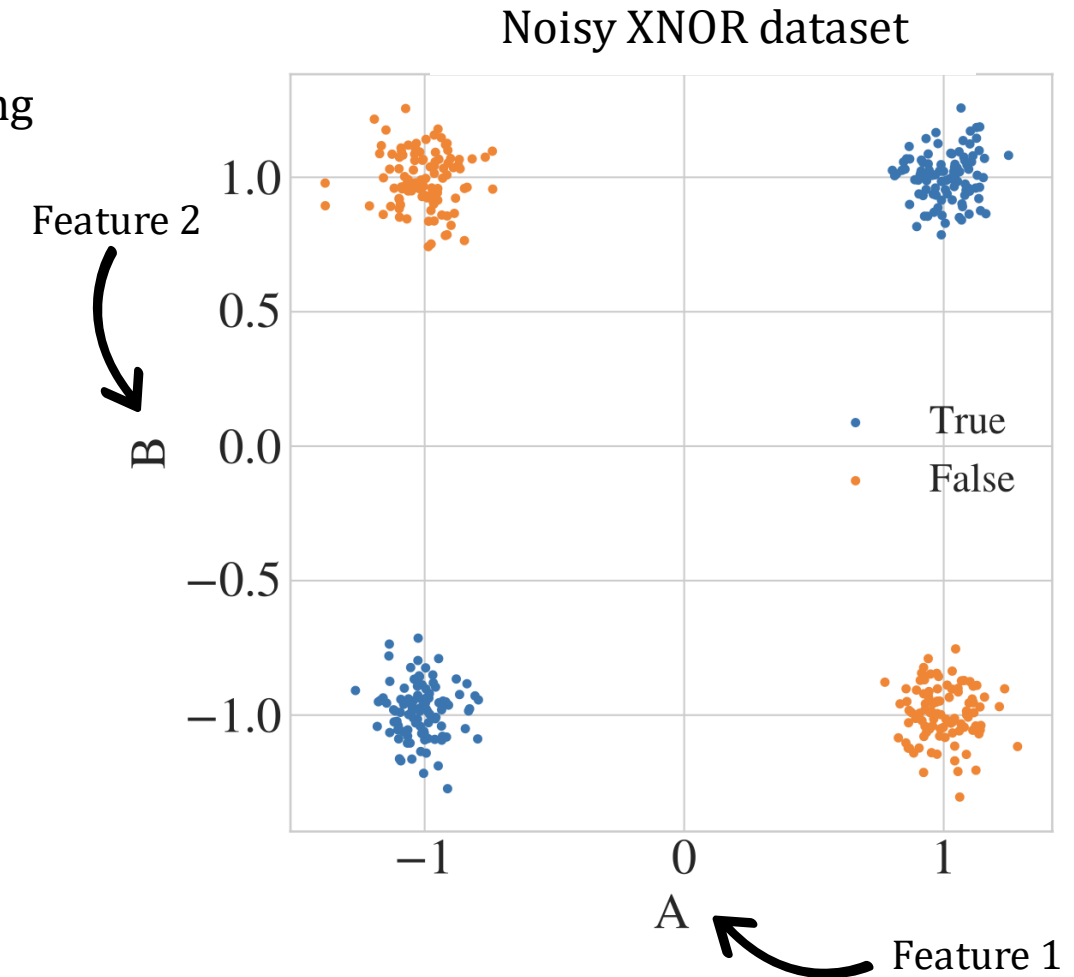
Decision trees

- Consider a **classification task** on an artificial dataset replicating the XNOR function

A	B	XNOR
True	True	True
True	False	False
False	True	False
False	False	True

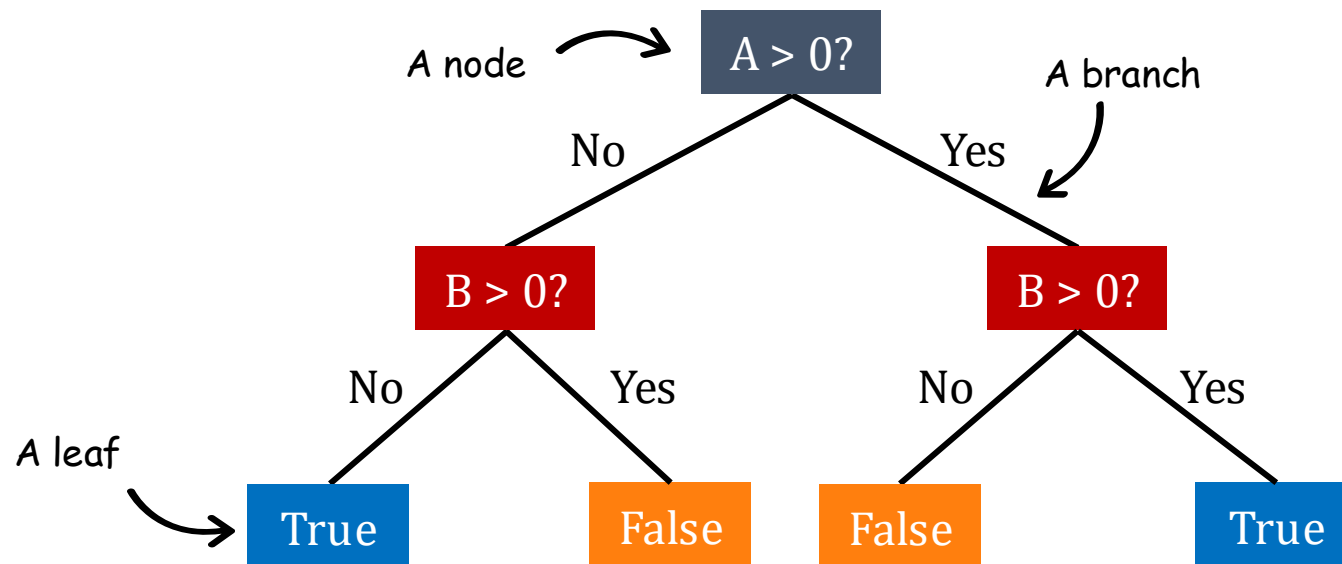
- Data are $n = 500$ couples $(x^{(i)}, y^{(i)}) \Rightarrow$ **Supervised learning**
- The target variable $y \in \{-1, 1\}$ is discrete $\Rightarrow$ **Classification**
- A linear classification would not be able to learn such a function*
- Decision trees** to the rescue!

*In fact, an alternative (not discussed here) would be to use **kernels** to find a higher dimensional embedding ϕ that makes the classes linearly separable and then use a linear classifier.



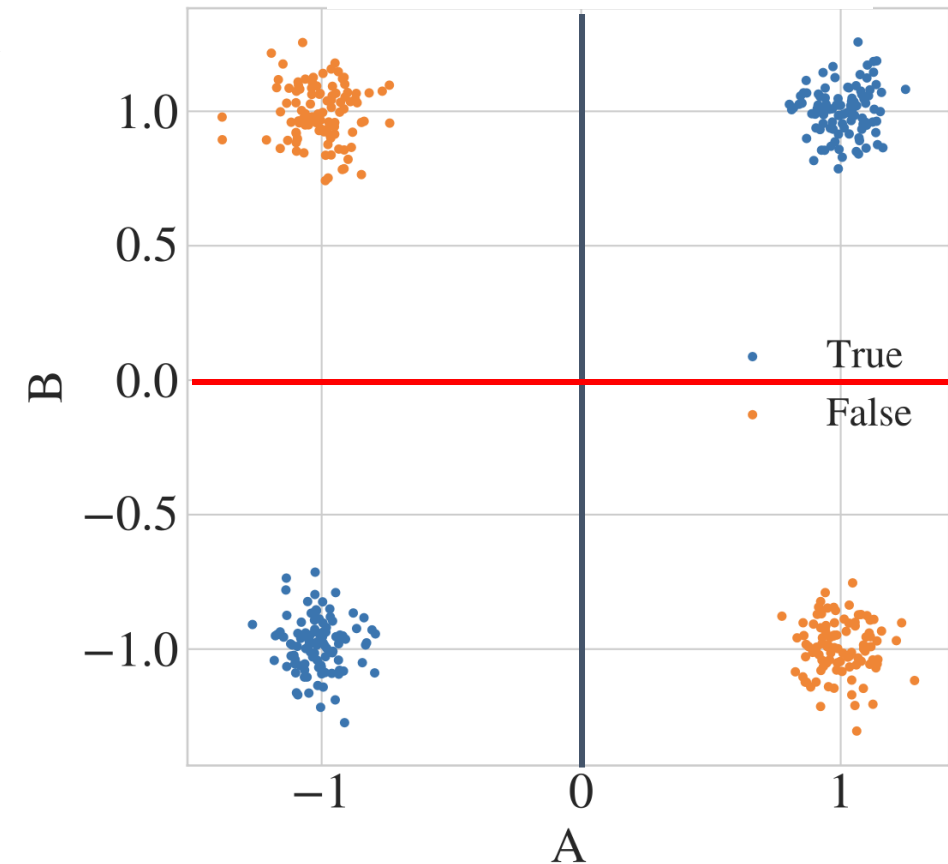
Decision trees

- Decision trees incrementally ask questions about the features to split the problem into smaller, simpler (binary) decisions



- All root and inner nodes question the value of a feature, and branches split the dataset into **different regions to which a datapoint can belong uniquely**

Noisy XNOR dataset



Decision trees

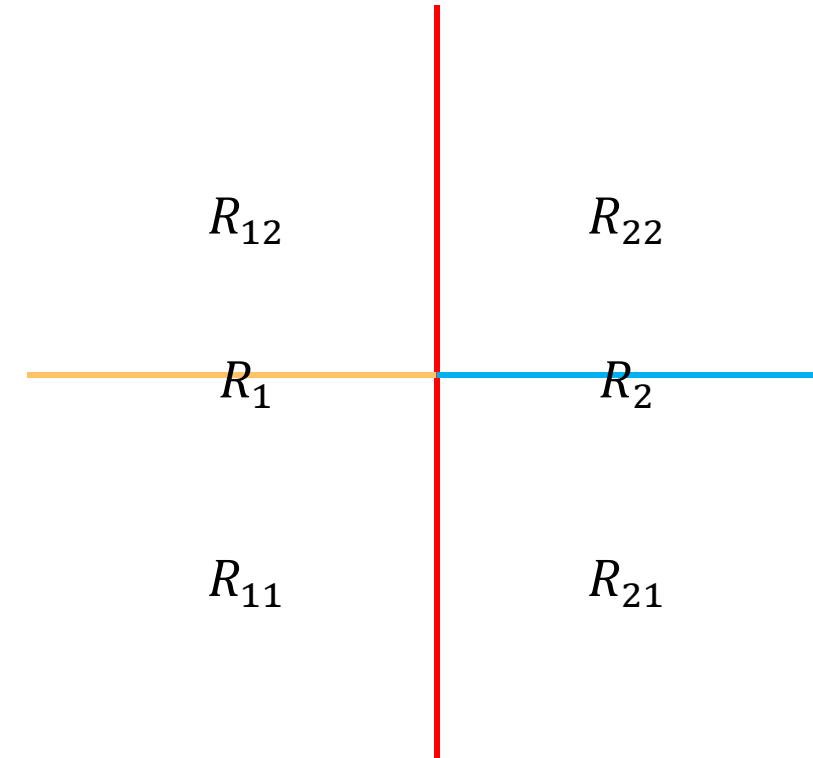
- More formally, at a given node in parent region R_i asking a question about the j^{th} feature, we create two regions:

$$R_{i1} = \{\mathbf{x} \mid x_j < \alpha_i^j, \mathbf{x} \in R_i\}$$

$$R_{i2} = \{\mathbf{x} \mid x_j \geq \alpha_i^j, \mathbf{x} \in R_i\}$$

- The parameters θ of decision trees are the threshold values at each nodes (the sequence of α)
- Decision tree minimise a criterion at each node of the tree

Noisy XNOR dataset



Decision trees

- To find the best possible parameters, decision trees maximize **the gain** through **a measure of “impurity” at each node until a stopping criterion is met** (depth of the tree, minimum number of samples in nodes)
- Popular error measures at a node are the **cross-entropy** (for classification) or the **squared error** (for regression)

Classification

$$\ell(R_i) = - \sum_{k=1}^q \rho_k^i \log_2 \rho_k^i, \quad \rho_k^i = \frac{|\{\mathbf{x}^{(j)} \mid \mathbf{x}^{(j)} \in R_i, y^{(j)} = k\}|}{|\{\mathbf{x}^{(j)} \mid \mathbf{x}^{(j)} \in R_i\}|}$$

Proportion of points in class k at node i

→ If the node is *pure*, $\rho_k^i = 1$ for a single class giving an entropy of 0, if the node is mixed with several classes, then entropy is large

Regression

$$\ell(R_i) = \frac{1}{N} \sum_{j=1}^N (y^{(j)} - m)^2, \quad m = \frac{1}{N} \sum_{\mathbf{x}^{(j)} \in R_i} y^{(j)}$$

Average value in the region

→ Takes the value in the region that minimizes the average squared error

Decision trees

Classification

$$\ell(R_i) = - \sum_{k=1}^q \rho_k^i \log_2 \rho_k^i,$$

$$\rho_k^i = \frac{|\{\mathbf{x}^{(j)} \mid \mathbf{x}^{(j)} \in R_i, y^{(j)} = k\}|}{|\{\mathbf{x}^{(j)} \mid \mathbf{x}^{(j)} \in R_i\}|}$$

Proportion of points in class k at node i

Regression

$$\ell(R_i) = \frac{1}{N} \sum_{j=1}^N (y_i - m)^2,$$

$$m = \frac{1}{N} \sum_{\mathbf{x}^{(j)} \in R_i} y^{(j)}$$

Average value in the region

- At each node, we look for the split into left/right regions maximizing the **gain** expressed in terms of a split

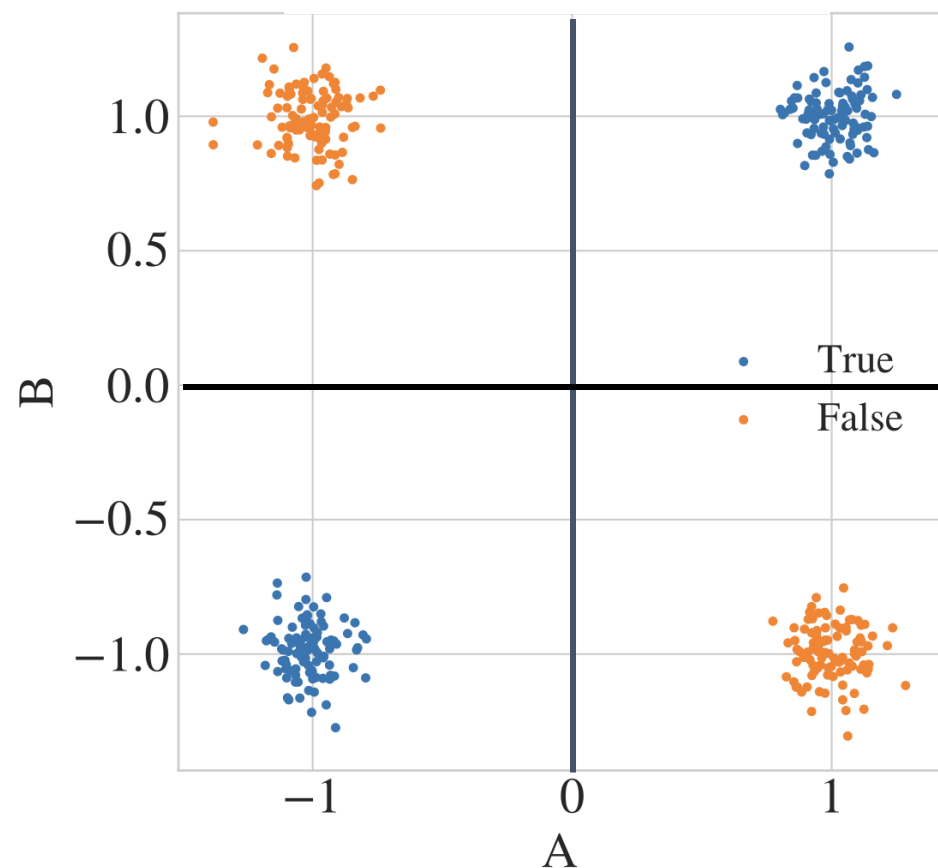
$$G_i = \ell(R_i) - \left(\frac{N_L}{N} \ell(R_{i1}) + \frac{N_R}{N} \ell(R_{i2}) \right)$$

N_L, N_R : number of datapoints in left/right regions

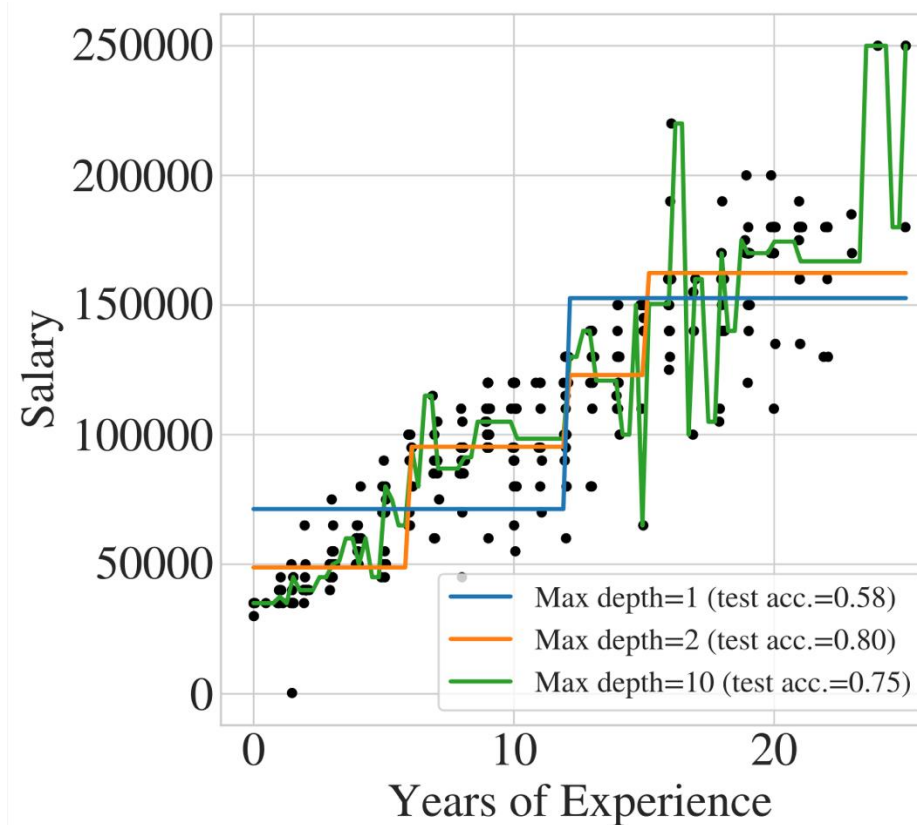
- +** can handle categorical values, easy to interpret, little data preparation, fast, both regression and classification
- **Shallow trees: high bias estimators (underfit), deep trees: high variance estimators (overfit)**

Decision trees

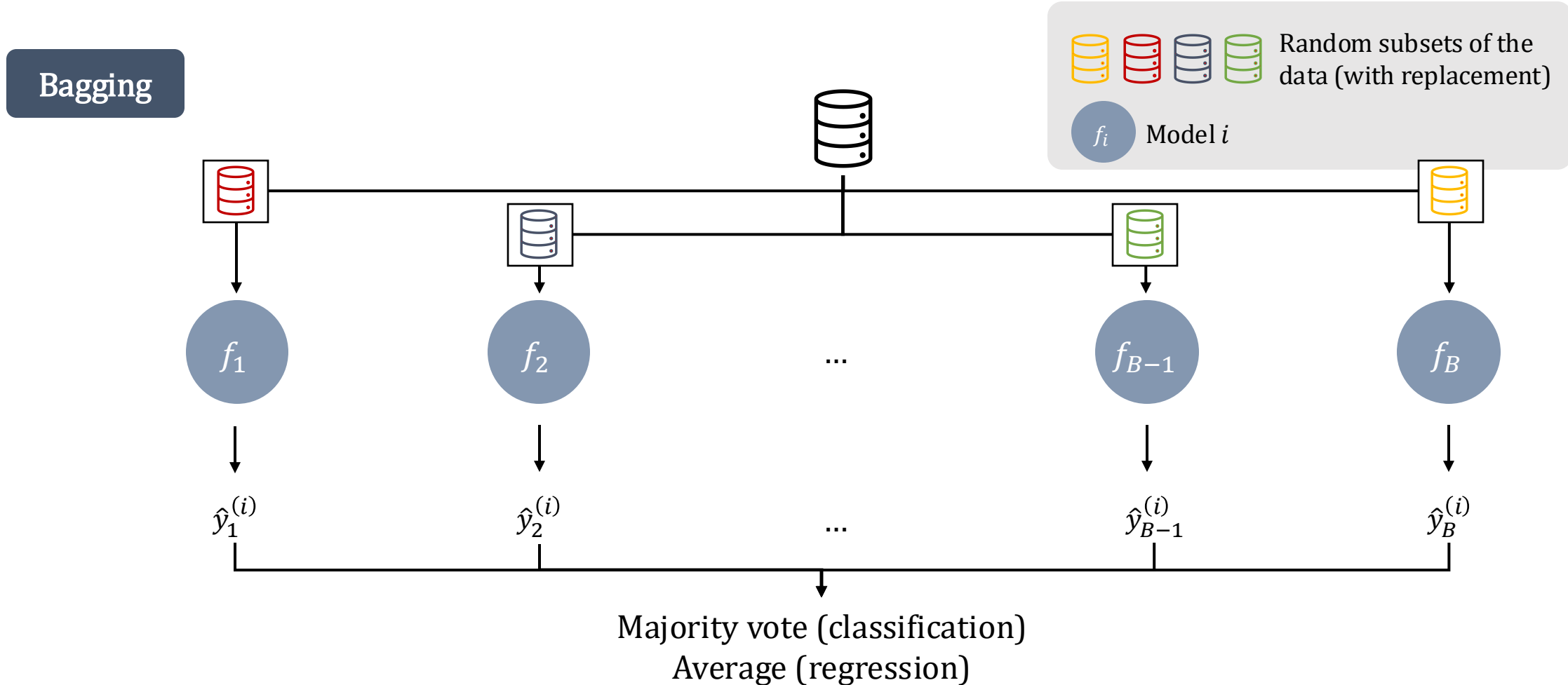
Classification
Noisy XNOR dataset



Regression
Salary prediction

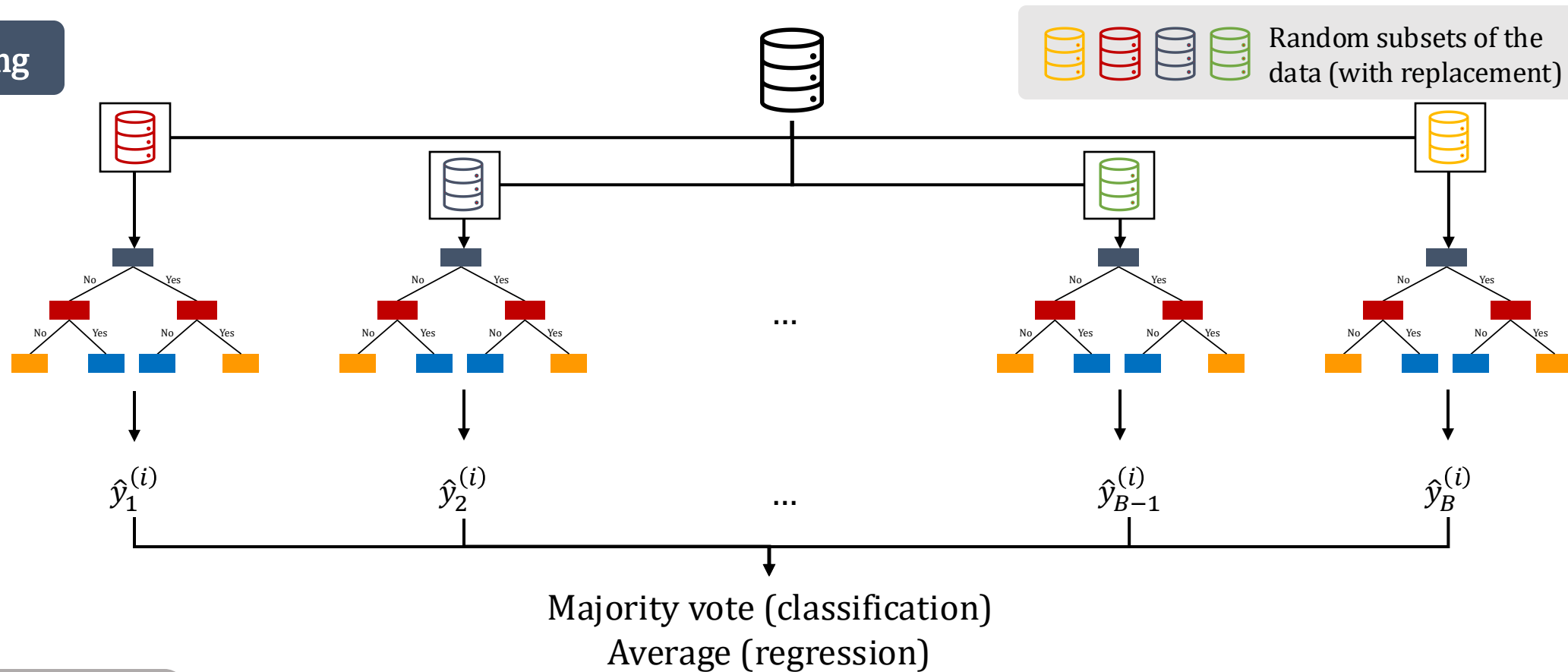


- To circumvent the problems that can have weak learners like decision trees, **ensembling methods** were proposed



- To reduce the variance, models need to be **uncorrelated**: this is achieved by using **random sampling of the dataset**

Bagging

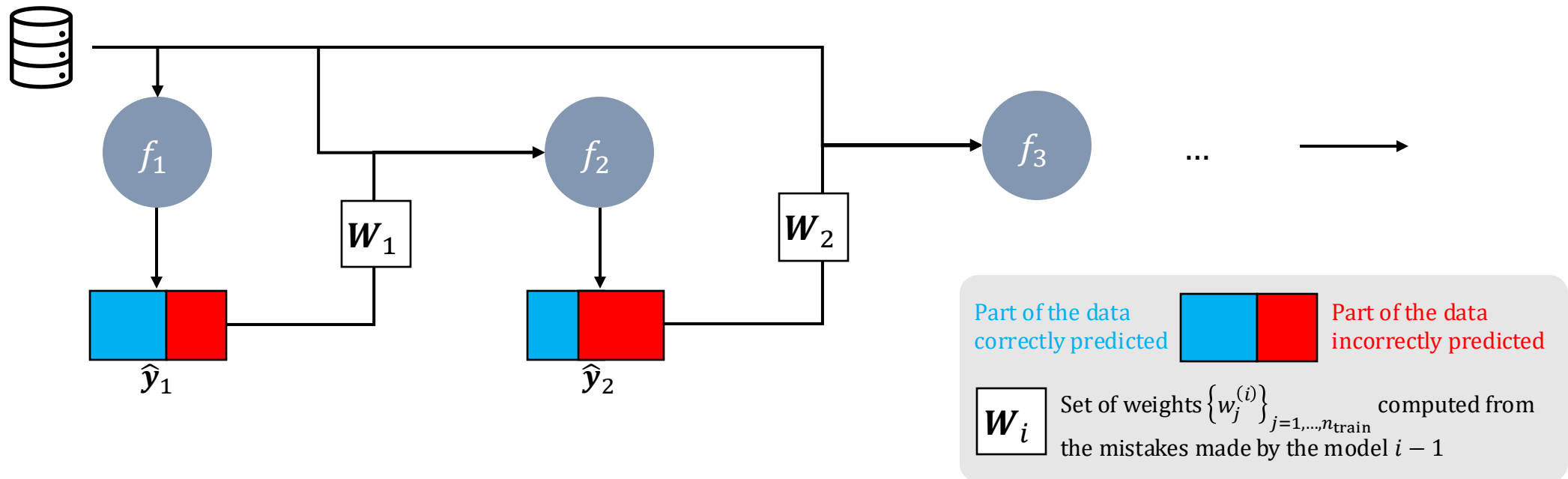


Random Forest = Bagging of decision trees + feature bagging

- + Can still handle categorical values, both regression and classification, **reduced variance**
- More expensive to compute (need to train B trees instead of one), harder to interpret

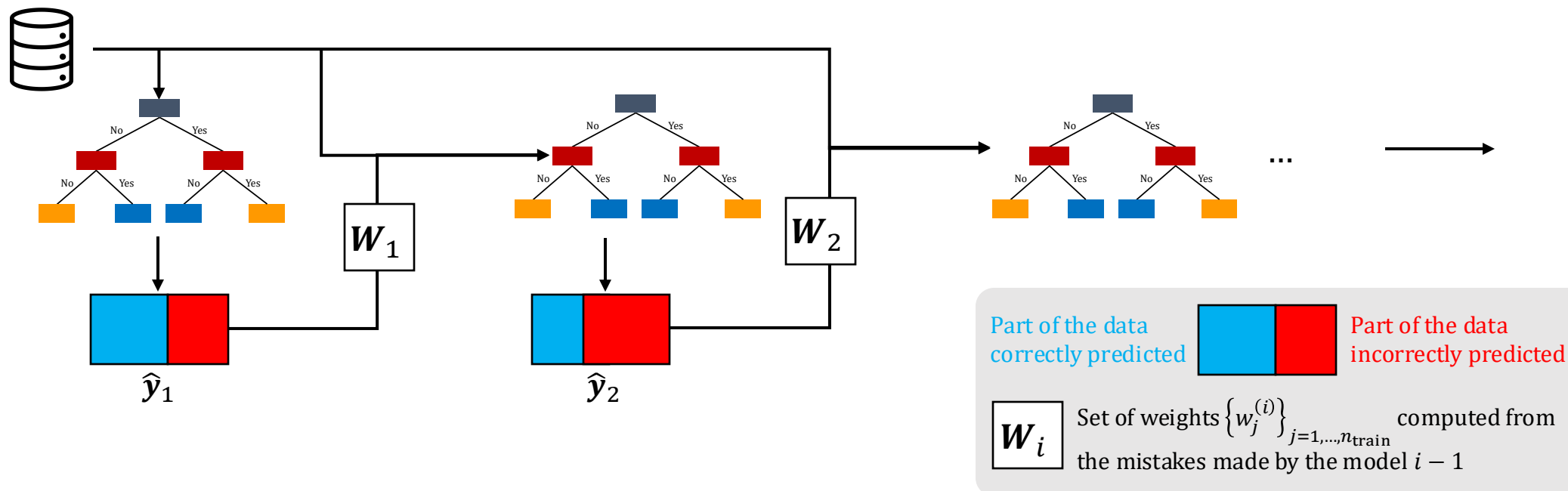
- While bagging trains high-variance models in parallel to reduce the variance of the combined estimate, **boosting trains high-bias models sequentially to reduce the overall bias**

Boosting



- Successive learners f_i are fed by data X_i , a weighted version of the initial dataset X , **giving more weights to the errors committed by the previous model f_{i-1}**
- The output is, as in bagging, **a linear combination of all the weak learners** weighted by the contribution of each tree
- The choice of weighting and training depends on the algorithm and context (see [Adaboost](#) or [gradient boosting](#))

Boosting



Boosted trees = Boosting of decision trees

- +** Both regression and classification (residuals or weighted classification error), **reduced bias**, good performances
- More expansive to compute, increased variance, subject to overfitting

Comparison on our regression problem

